Computer Forensics Case Studies

Case Study 3 Investigating Memory Evidence

Student Assessment Brief

HUSSEINI SULEIMAN

2025/FWSD/11478

fwsd2511478@students.icdfa.edu.ng

DATE: 19Th October, 2026

Assessment detail	Information
Course	SBT-DF204 Computer Forensics Case Studies
Assessment	Case Study 3 Individual memory forensics investigation
Case title	Investigating Suspected Illegal File Transfer From Memory Evidence
Submission format	Written report with Volatility output, screenshots, timeline, and evidence log
Deadline and maximum score	As published in E-Campus

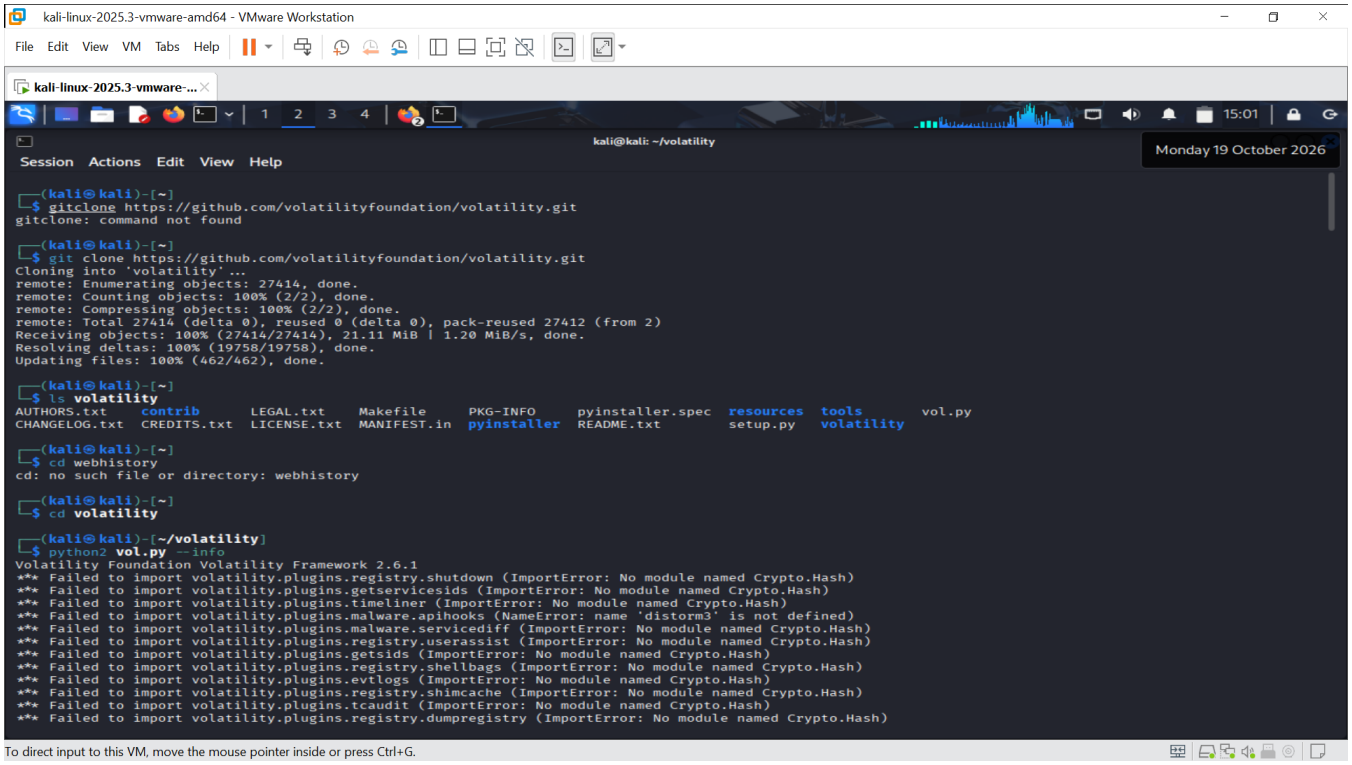
This individual case study asks you to examine a Windows 7 memory image and reconstruct an evidence-based account of a suspected illegal file transfer. Use Volatility 2 to identify relevant processes, network artefacts, command history, removable-media records, user-profile evidence, and File Explorer traces. Your report must clearly separate direct observations from inferences about a transfer or the person who used the workstation.

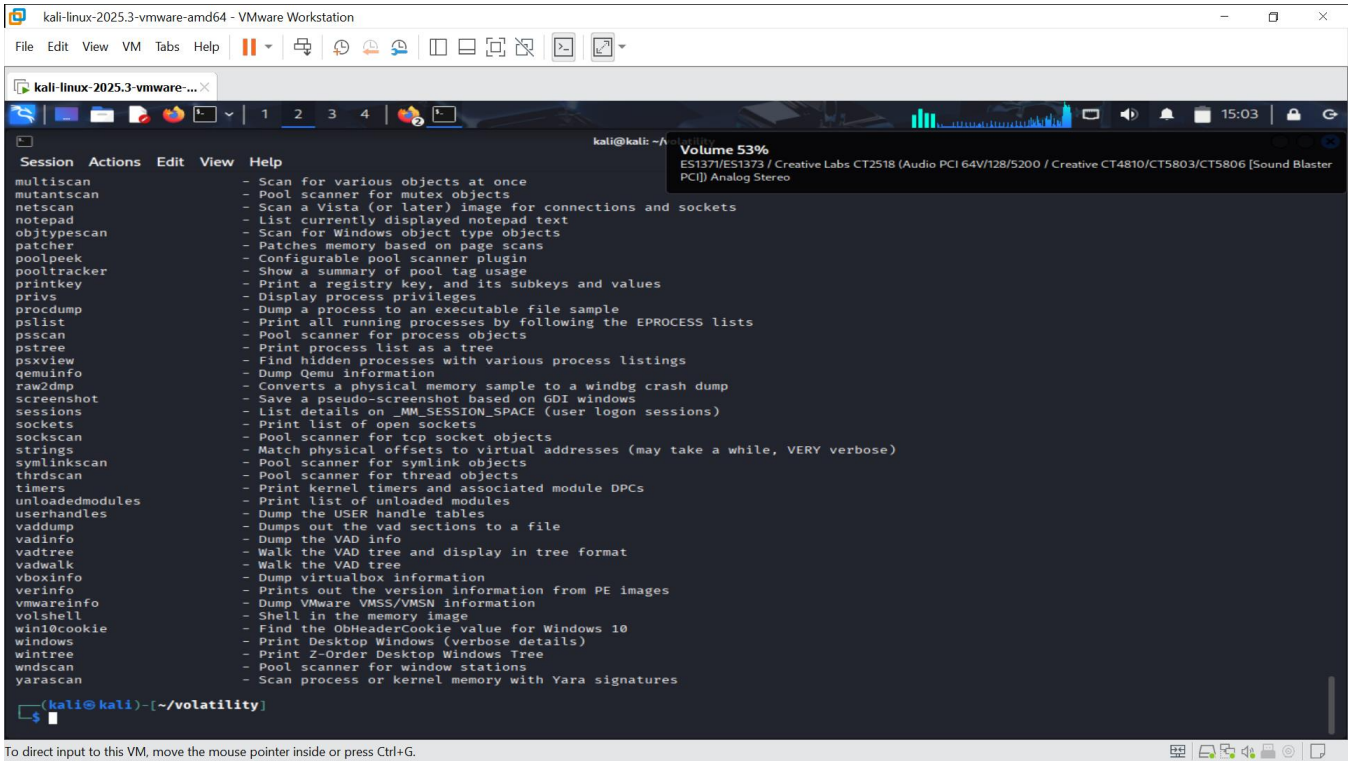
Case scenario

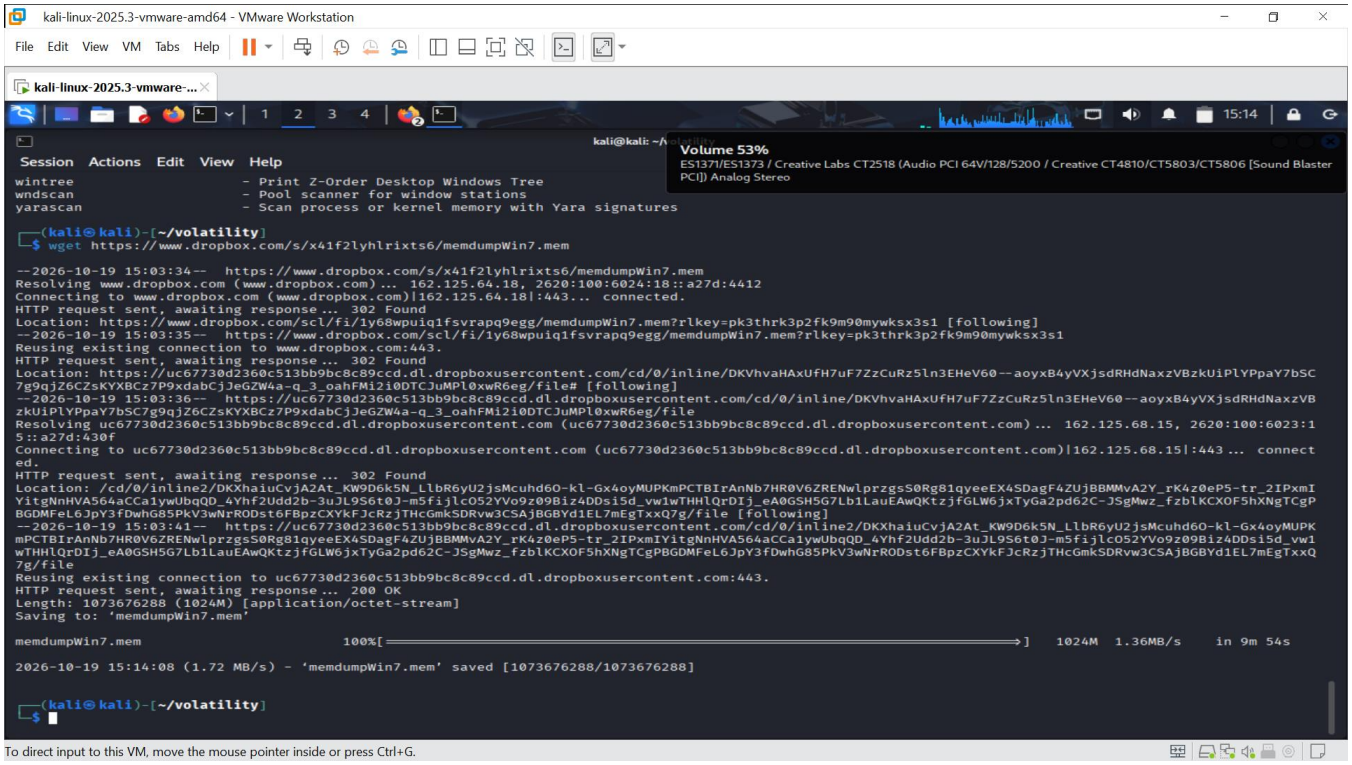
Investigators obtained a Windows 10 memory image from a workstation after a confidential document may have been transferred without authorization. The inquiry is whether browser activity, a command prompt, a removable device, and a user profile form a defensible sequence. This is an offline examination of the supplied material. It does not authorize contact with any address, host, account, file-sharing service, or person that may appear in the image.

Case materials

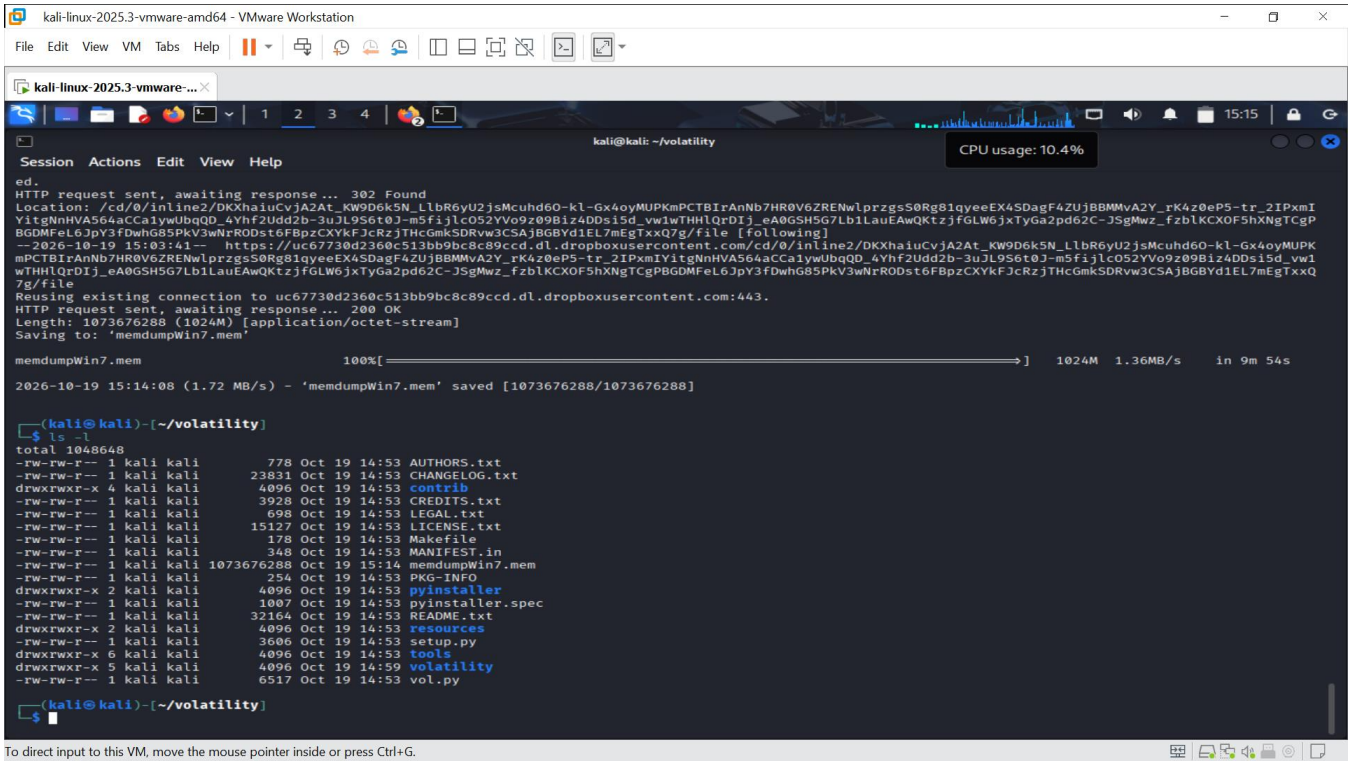
The supplied case file is a Windows 7 memory image named memdumpWin7.mem. The teaching deck demonstrates Volatility 2 methods. In the ICDFA Kali lab, start Volatility 2 with python2 vol.py. Create a working copy before analysis. Record the source, original file name, file size, SHA-256 value, and tool version. Select and justify the memory profile from the actual image output instead of assuming one from a screenshot.


kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-...
15:01
kali@kali: ~/volatility
Session Actions Edit View Help
Monday 19 October 2026
┌──(kali㉿kali)-[~]
└─$ gitclone https://github.com/volatilityfoundation/volatility.git
gitclone: command not found
┌──(kali㉿kali)-[~]
└─$ git clone https://github.com/volatilityfoundation/volatility.git
Cloning into 'volatility' ...
remote: Enumerating objects: 27414, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 27414 (delta 0), reused 0 (delta 0), pack-reused 27412 (from 2)
Receiving objects: 100% (27414/27414), 21.11 MiB | 1.20 MiB/s, done.
Resolving deltas: 100% (19758/19758), done.
Updating files: 100% (462/462), done.
┌──(kali㉿kali)-[~]
└─$ ls volatility
AUTHORS.txt contrib LEGAL.txt Makefile PKG-INFO pyinstaller.spec resources tools vol.py
CHANGELOG.txt CREDITS.txt LICENSE.txt MANIFEST.in pyinstaller README.txt setup.py volatility
┌──(kali㉿kali)-[~]
└─$ cd webhistory
cd: no such file or directory: webhistory
┌──(kali㉿kali)-[~]
└─$ cd volatility
┌──(kali㉿kali)-[~/volatility]
└─$ python2 vol.py --info
Volatility Foundation Volatility Framework 2.6.1
*** Failed to import volatility.plugins.registry.shutdown (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.getservicesids (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.timeliner (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.malware.apihooks (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.malware.servicediff (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.userassist (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.getsids (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shellbags (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.evtlogs (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.tcaudit (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.dumpregistry (ImportError: No module named Crypto.Hash)
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

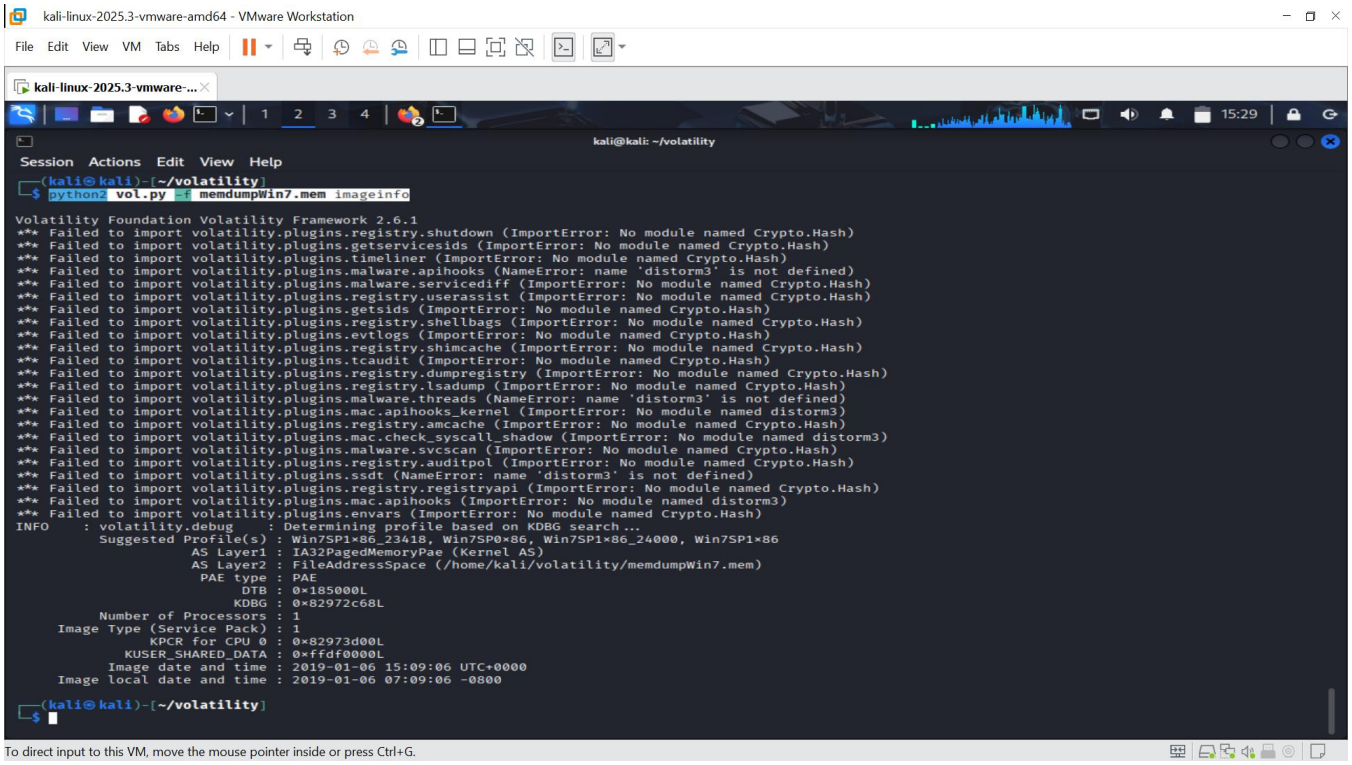

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-...
15:03
Session Actions Edit View Help
Volume 53%
ES1371/ES1373 / Creative Labs CT2518 (Audio PCI 64V/128/5200 / Creative CT4810/CT5803/CT5806 [Sound Blaster PCI]) Analog Stereo
multiscan - Scan for various objects at once
mutantscan - Pool scanner for mutex objects
netscan - Scan a Vista (or later) image for connections and sockets
notepad - List currently displayed notepad text
objtypescan - Scan for Windows object type objects
patcher - Patches memory based on page scans
poolpeek - Configurable pool scanner plugin
pooltracker - Show a summary of pool tag usage
printkey - Print a registry key, and its subkeys and values
privs - Display process privileges
procdump - Dump a process to an executable file sample
pslist - Print all running processes by following the EPROCESS lists
psscan - Pool scanner for process objects
pstree - Print process list as a tree
psxview - Find hidden processes with various process listings
qemuinfo - Dump Qemu information
raw2dmp - Converts a physical memory sample to a windbg crash dump
screenshot - Save a pseudo-screenshot based on GDI windows
sessions - List details on _MM_SESSION_SPACE (user logon sessions)
sockets - Print list of open sockets
sockscan - Pool scanner for tcp socket objects
strings - Match physical offsets to virtual addresses (may take a while, VERY verbose)
symlinkscan - Pool scanner for symlink objects
thrdscan - Pool scanner for thread objects
timers - Print kernel timers and associated module DPCs
unloadedmodules - Print list of unloaded modules
userhandles - Dump the USER handle tables
vaddump - Dumps out the vad sections to a file
vadinfo - Dump the VAD info
vadtree - Walk the VAD tree and display in tree format
vadwalk - Walk the VAD tree
vboxinfo - Dump virtualbox information
verinfo - Prints out the version information from PE images
vmwareinfo - Dump VMware VMSS/VMSN information
volshell - Shell in the memory image
win10cookie - Find the ObHeaderCookie value for Windows 10
windows - Print Desktop Windows (verbose details)
wintree - Print Z-Order Desktop Windows Tree
wndscan - Pool scanner for window stations
yarascan - Scan process or kernel memory with Yara signatures
┌──(kali㉿kali)-[~/volatility]
└─$
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.



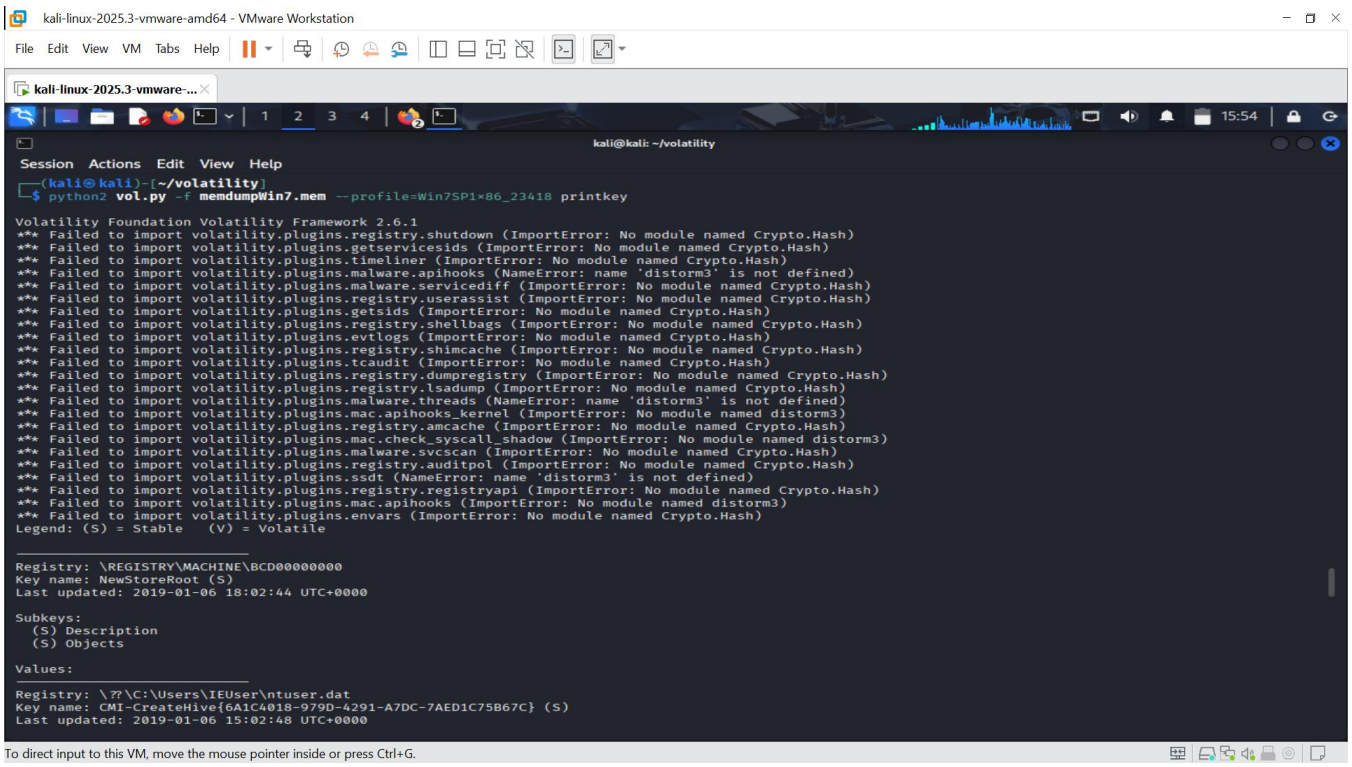
Payload used: wget https://www.dropbox.com/s/x41f2lyhlrixts6/memdumpWin7.mem

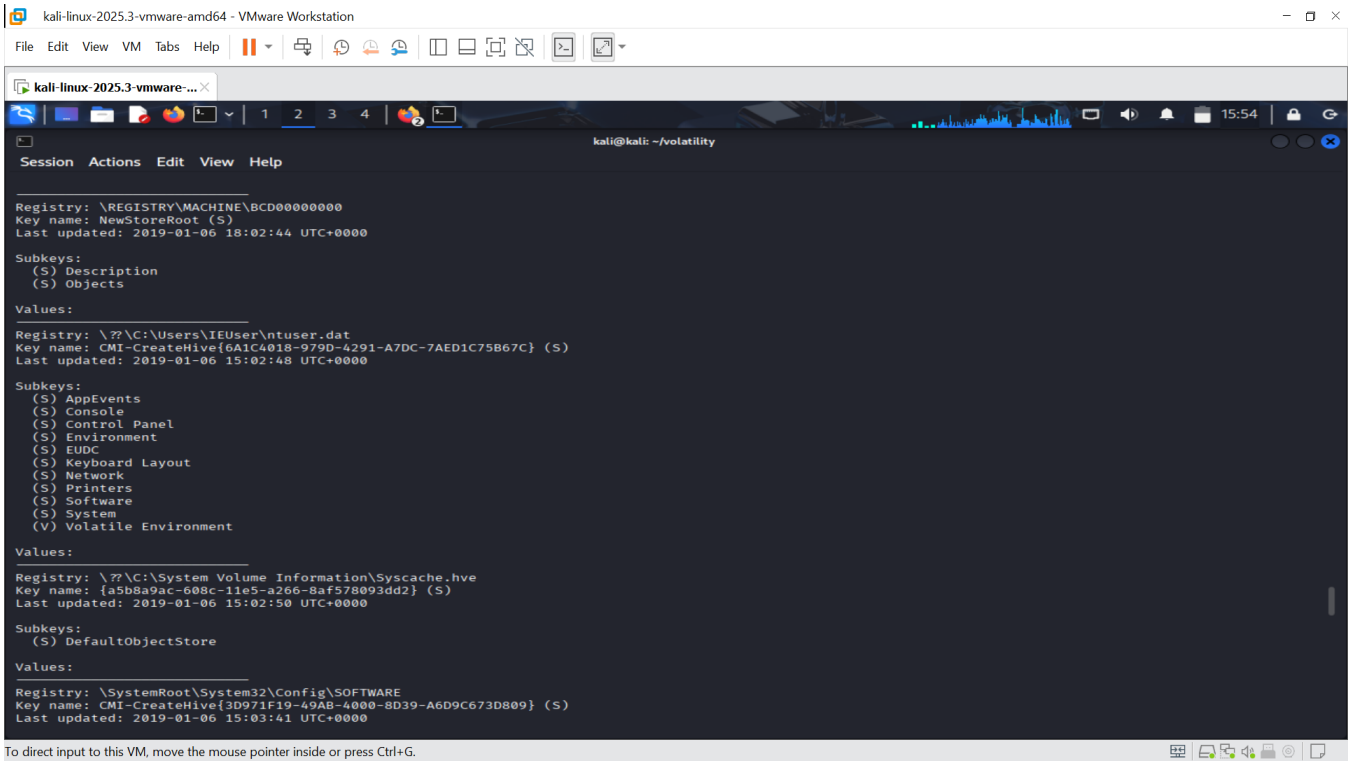


Payload used: python2 vol.py -f memdumpWin7.mem imageinfo

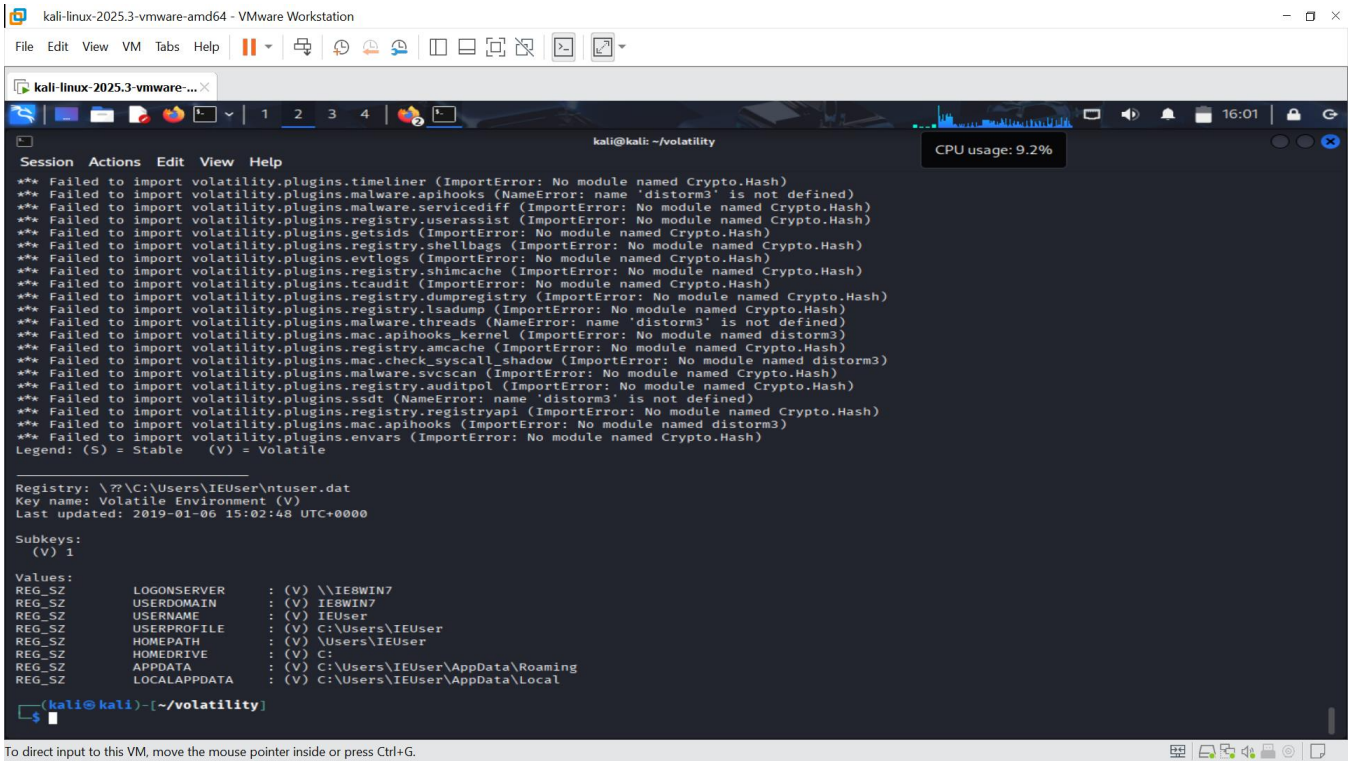


Command used: python2 vol.py -f memdumpWin7.mem --profile=Win7SP1x86_23418 printkey

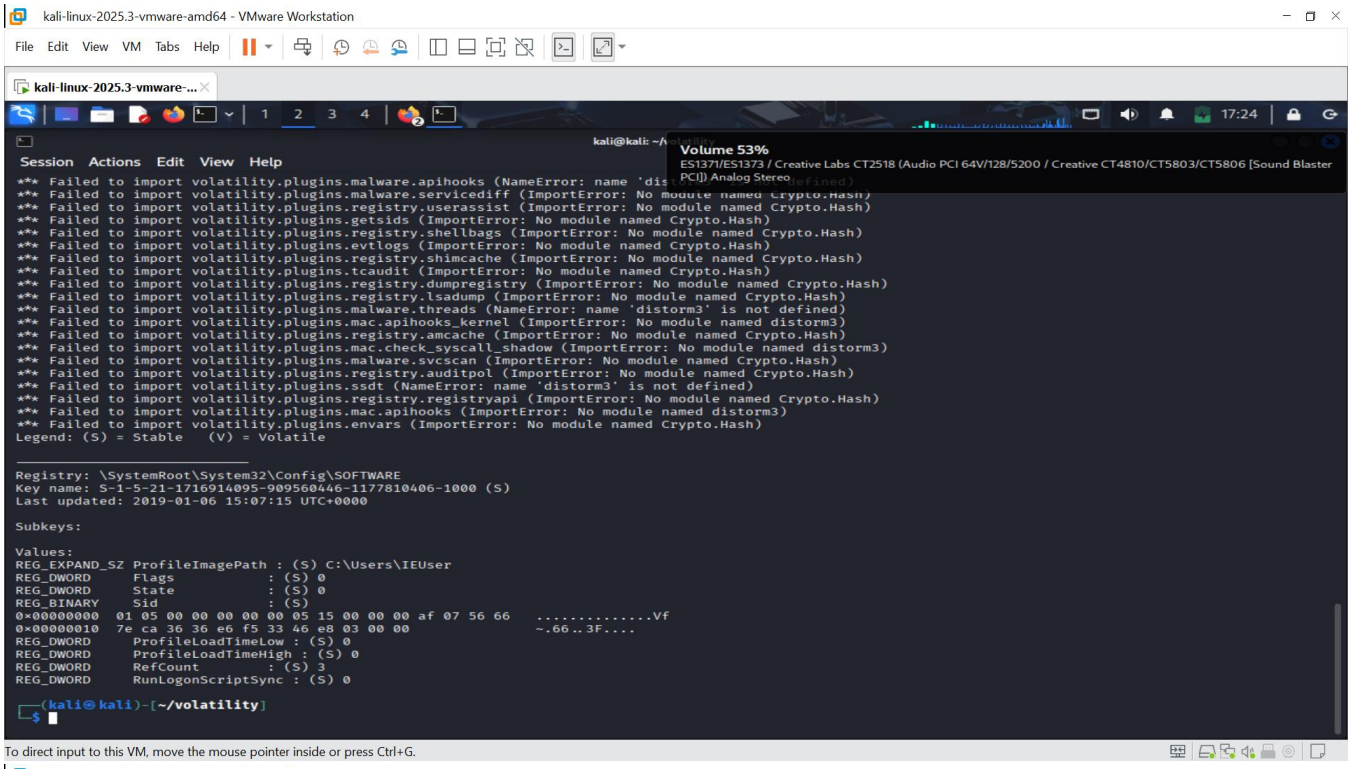


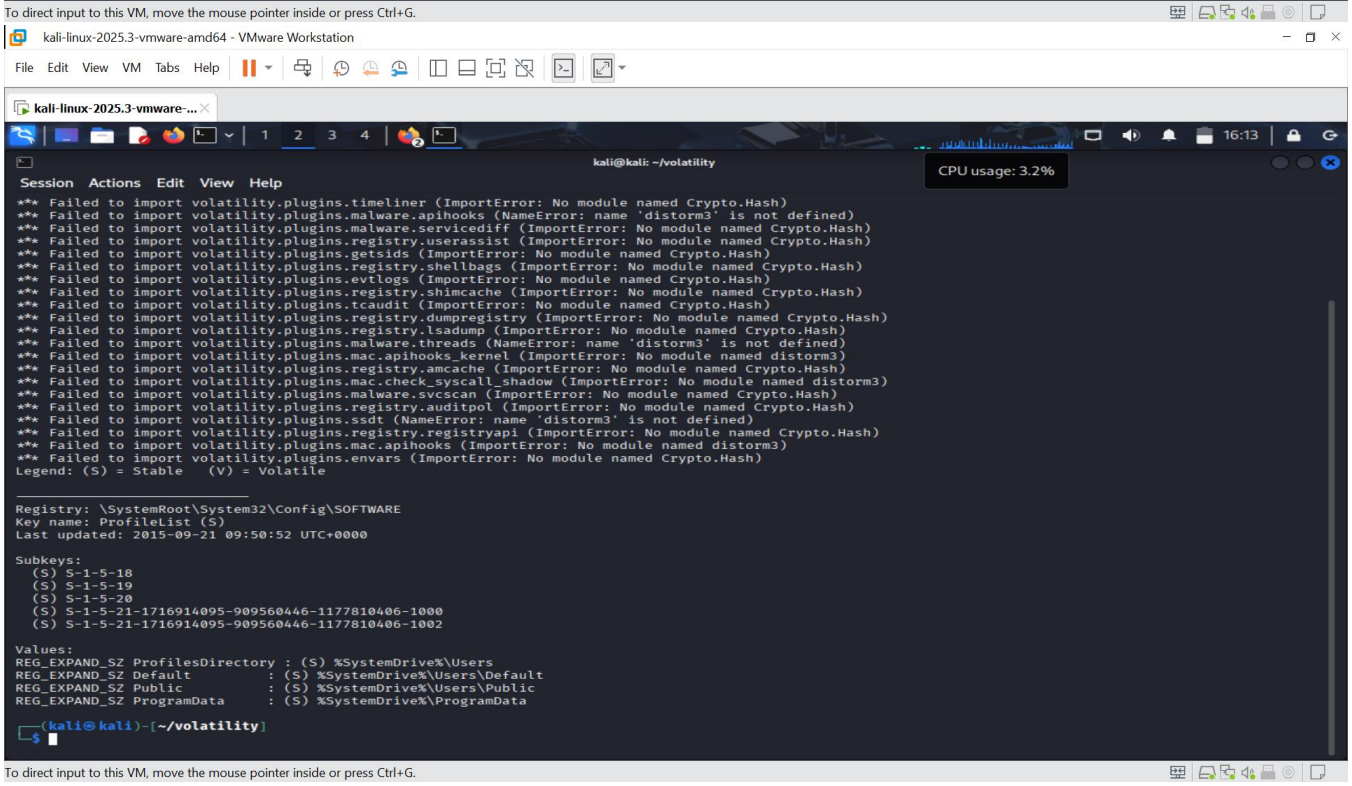


Command used: vol.py -f memdumpWin7.mem --profile=Win7SP1x86_23418 printkey -K "Volatile Environment"

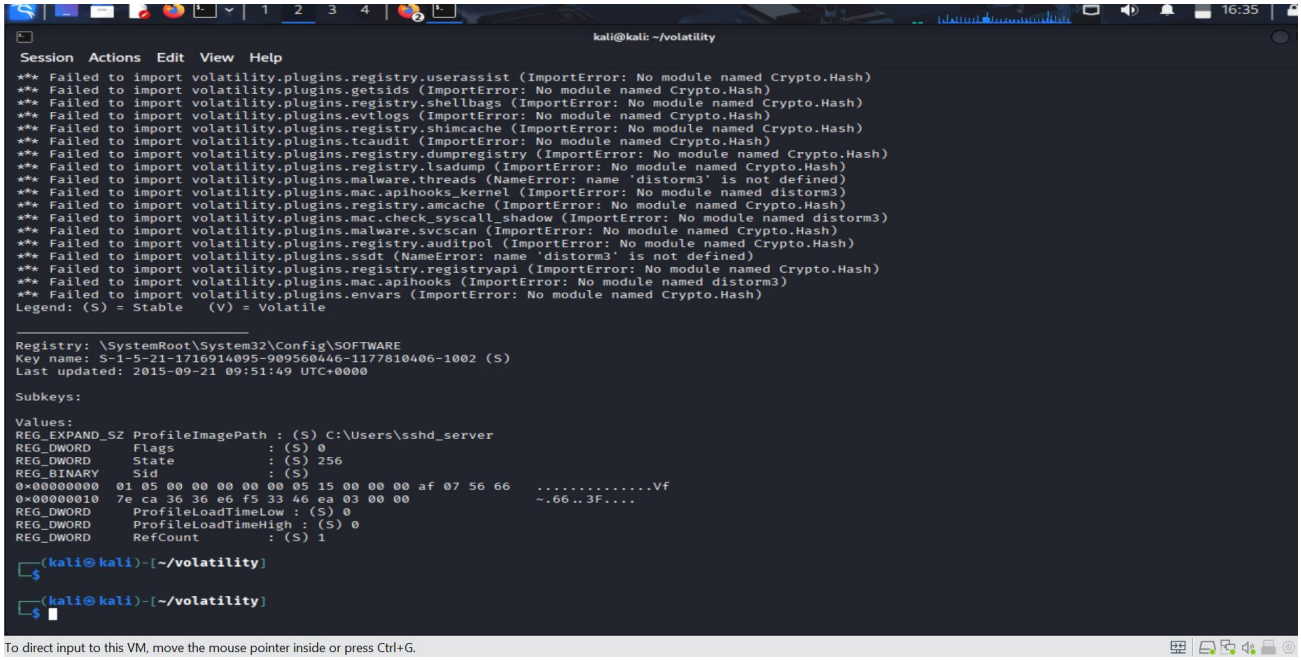


Command used: python2 vol.py -f memdumpWin7.mem --profile=Win7SP1x86_23418 printkey -K "Microsoft\Windows NT\CurrentVersion\ProfileList\S-1-5-21-1716914095-909560446-1177810406-1000"





Command used: python2 vol.py -f memdumpWin7.mem --profile=Win7SP1x86_23418 printkey -K "Microsoft\Windows NT\CurrentVersion\ProfileList\S-1-5-21-1716914095-909560446-1177810406-1002"



Command used: python2 vol.py -f memdumpWin7.mem --profile=Win7SP1x86_23418 printkey -K "Microsoft\Windows NT\CurrentVersion\ProfileList\S-1-5-21-1716914095-909560446-1177810406-1000"

What you must submit

Item	Required output	What to provide
1	Forensic working copy	Acquire the supplied memory image. Record its source, original file name, byte size, SHA-256 value, and the location of your read-only original and working copy.
2	Volatility evidence set	Preserve output from profile identification, process and network analysis, command history, USB registry and MountPoints2 checks, and ShellBag analysis. Label every output with the plugin, profile, and command used.
3	Reconstructed timeline	Build a time-ordered account of the suspected transfer. Cite process IDs, registry paths, offsets or record details, timestamps, and evidence-log IDs where relevant.
4	Student report	Submit one report in PDF or DOCX format with labelled screenshots, a completed evidence log, and a timeline appendix.

Investigation task

Examine the memory image methodically. Base every finding on identifiable Volatility output, including the plugin, profile, command, timestamp, record, registry path, process ID, or offset that supports it. A recovered string, process name, command, or registry key can be important, but it may not prove that a transfer completed or that a particular person performed it. Use the questions below to structure your report.

No	Investigation question	Minimum evidence to discuss
1	What evidence image did you acquire and how did you preserve it?	Record the source, file name, file size, SHA-256 value, working-copy method, file-access precautions, and tools used.
2	What operating-system profile and image time are supported?	Provide imageinfo output, the candidate and selected profile, the image time, and the time zone. Explain how you validated the profile.
3	What user and account artefacts are relevant?	Identify the account or profile records examined, their SIDs or registry-hive paths, and the limit between profile evidence and proof of a person at the keyboard.

No	Investigation question	Minimum evidence to discuss
4	What process and network evidence relates to a potential download or transfer?	Identify relevant process IDs, process names, local and foreign endpoints, connection state, and memory strings or output records. Do not contact any endpoint.
5	What does command history show?	Use cmdscan or consoles output to identify relevant commands, the command process, path or drive references, and the timing or correlation evidence. Explain any limit on proving success.
6	What removable-media evidence is present?	Use USBSTOR, DeviceClasses, MountedDevices, and MountPoints2 evidence as available to identify device, drive-letter, and user-profile relationships. State what each artefact does and does not establish.
7	What File Explorer evidence supports the sequence?	Use ShellBags to identify material folder paths and recorded access or preference times. Distinguish folder-view evidence from a file-modification timestamp.
8	What conclusion can you defend?	Create a UTC timeline and state a cautious conclusion with a confidence level and at least three limitations or alternative explanations.

Volatility 2 examination starting points

Use the commands below as starting points. Replace <profile> with the profile you select from imageinfo and record any variation needed for the authorised lab environment. The commands guide your examination; the returned records provide the evidence.

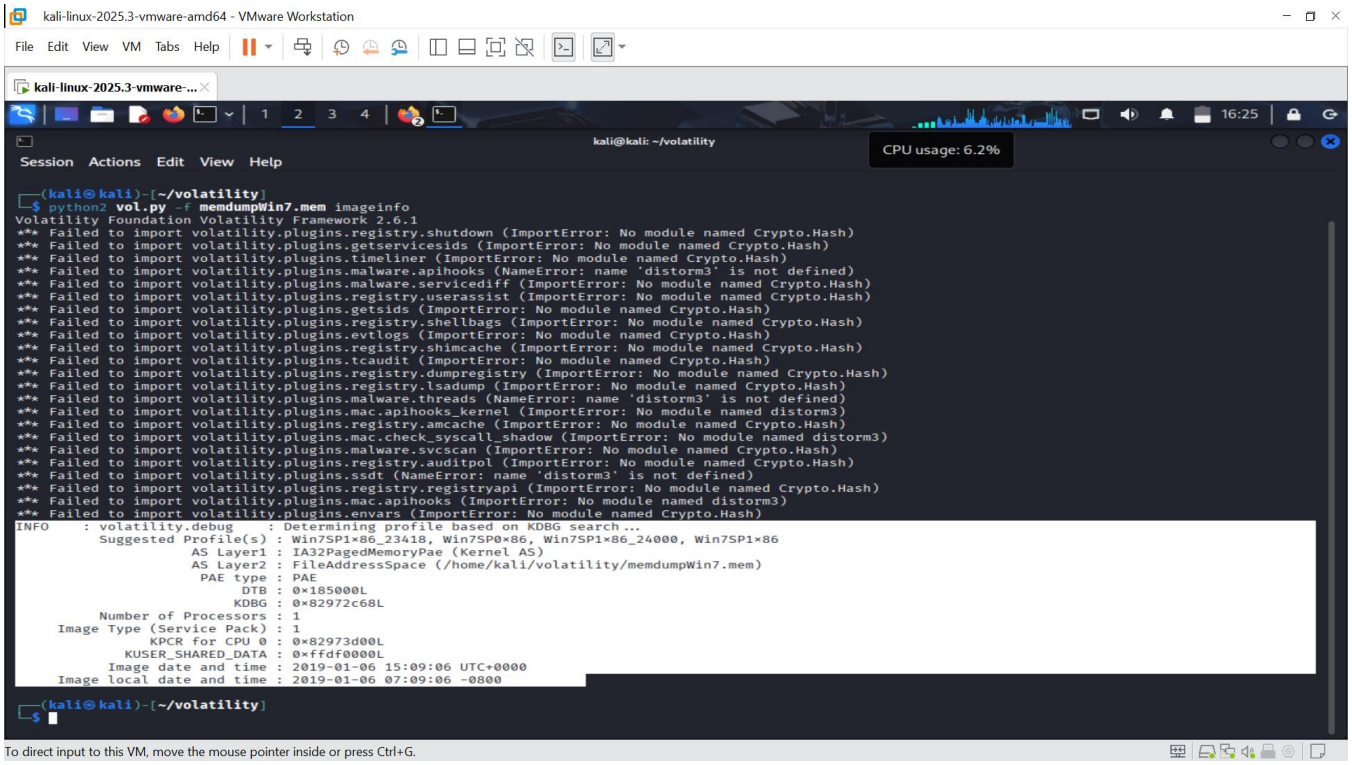
Purpose	Starting command
Identify the image profile and capture time	`python2 vol.py -f memdumpWin7.mem imageinfo` **Image profile:** Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86 Captured time: 2019-01-06 15:09:06 UTC+0000 Image local date and time : 2019-01-06 07:09:06 -0800

List active and residual processes	`python2 vol.py -f memdumpWin7.mem --profile=<profile> pslist`
	`python2 vol.py -f memdumpWin7.mem --profile=<profile> psscan`
	`Proceses`

Review network artefacts	`python2 vol.py -f memdumpWin7.mem --profile=<profile> netscan`
Review command history	`python2 vol.py -f memdumpWin7.mem --profile=<profile> cmdscan` `python2 vol.py -f memdumpWin7.mem --profile=<profile> consoles`

Locate USB and drive-mapping artefacts	`python2 vol.py -f memdumpWin7.mem --profile=<profile> hivelist`
	`python2 vol.py -f memdumpWin7.mem --profile=<profile> printkey -K "ControlSet001\Enum\USBSTOR"`
	`python2 vol.py -f memdumpWin7.mem --profile=<profile> printkey -K "Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2"`

Review File Explorer history	`python2 vol.py -f memdumpWin7.mem --profile=<profile> shellbags --output=html --output-file=shellbags.html`



Evidence interpretation safeguards

- Validate a profile suggested by imageinfo with usable output from multiple relevant plugins and record the profile you selected.
- Treat network endpoints and process ownership as artefacts to corroborate. A closed connection or an unidentified PID needs careful interpretation.
- Assess a command-history entry with its process, timing, and related artefacts. A command string alone may not demonstrate that the copy operation completed.
- Use USBSTOR, MountedDevices, DeviceClasses, and MountPoints2 records to explain device, drive, and profile relationships. These records alone may not prove physical possession or a user identity at a particular time.
- Treat ShellBag times as folder-view or preference evidence unless other artefacts establish a different event.

Required report structure

1. Case reference and student identification. Include your full name, registration number, course code, and case title.
2. Executive conclusion. State the supported conclusion in a short, cautious paragraph and include a confidence level.
3. Evidence acquisition and integrity. Record the memory-image source, file name, size, hash, working-copy method, and tools used.
4. Method and profile selection. Explain the imageinfo result, selected profile, Volatility version, plugin commands, and time-zone treatment.
5. Findings. Answer each investigation question with Volatility output, registry paths, process IDs, offsets or records, timestamps, and evidence-log IDs.
6. Timeline. Present the material sequence in UTC and distinguish each observed artefact from your interpretation of its meaning.
7. Attribution assessment. Explain limitations, including memory-capture timing, profile versus human identity, command success, shared access, and incomplete or overwritten evidence.

8. Appendices. Include labelled screenshots, selected Volatility output, the completed timeline table, and the completed evidence log.

Timeline template

Use a completed version of this table in your report or evidence appendix. Add rows for all material events and keep the main timeline in UTC.

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
Browser or process evidence					
Download-folder review					
Removable-device evidence					
Drive mapping					
Command-history evidence	`python2 vol.py -f memdumpWin7.mem --profile=<profile> pslist`	`python2 vol.py -f memdumpWin7.mem --profile=<profile> netscan`	`python2 vol.py -f memdumpWin7.mem --profile=<profile> cmdscan` `python2 vol.py -f memdumpWin7.mem --profile=<profile> consoles`	`python2 vol.py -f memdumpWin7.mem --profile=<profile> psscan` `Proceses`	
Supported conclusion					

Evidence log template

Use an evidence log that lets an assessor locate the exact Volatility plugin, command, output, registry path, process record, or screenshot supporting each finding.

Evidence ID	Tool and plugin	Source record or path	Observed fact	Screenshot or appendix reference
E01				
E02				
E03				
E04				
E05				
E06				

Submission rules

- Work individually. Your Volatility analysis, report wording, screenshots, timeline, and evidence log must be your own.
- Submit the report as SBTDF204_CaseStudy3_RegistrationNumber.pdf or .docx. If you submit separate supporting material, use SBTDF204_CaseStudy3_Evidence_RegistrationNumber.zip.

- Do not upload the original memory image unless your lecturer specifically requests it. Submit only the report and the required supporting evidence.
- Use the image only as authorised course material. Do not contact remote IP addresses, hosts, accounts, services, or people named in the output. Do not execute recovered files or use password or hash-cracking steps in this case study.
- Redact unrelated personal data that may appear in screenshots while retaining enough artefact detail for the examiner to verify your finding.

Marking guide

The E-Campus maximum score applies to this case study. The assessor will apply the following percentage weighting to that maximum score.

Criterion	What strong work demonstrates	Weight
Evidence acquisition and profile validation	Records integrity, identifies a suitable profile, and explains the image time and time zone accurately.	15%
Process and network analysis	Uses process and network evidence carefully, identifies relevant artefacts, and distinguishes direct observations from interpretations.	20%
Command and transfer evidence	Examines command history, browser or process evidence, and the possible transfer sequence with defensible supporting output.	20%
USB, account, and registry correlation	Correlates removable-media, drive-mapping, user-profile, and registry evidence while explaining each artefact limit.	20%
Timeline, conclusion, and limitations	Builds a clear UTC timeline, reaches a cautious conclusion, and explains alternative explanations or gaps in evidence.	15%
Report quality and evidence presentation	Writes clearly, labels Volatility output and screenshots, cites evidence IDs, and submits a usable evidence log.	10%

Final check before submission

- Every major conclusion cites the Volatility plugin, command, selected profile, record or offset, timestamp, and screenshot or appendix reference.
- Your timeline uses UTC consistently and records any local-time conversion separately.
- Your report explains how process, network, command, USB, registry, and ShellBag artefacts support or challenge the proposed sequence.
- Your conclusion distinguishes a possible transfer attempt from proof of a completed transfer and separates workstation-profile activity from proof of a named person.